Zofia Kapcińska 160227

Sprawozdanie: Implementacja algorytmu Diffiego-Hellmana uzgadniania klucza sesyjnego

1. Screenshot.

```python
import hashlib
import time
import random
import string
import numpy as np

def generateHash(text, algorithm):  4 usages
    hashObj = hashlib.new(algorithm)
    hashObj.update(text)
    return hashObj.hexdigest()

def findCollision(algorithm='sha256'):  1 usage
    hexNum = 12 // 4
    seen = {}
    attemps = 1

    while 1:
        randomText = ''.join(random.choices(string.ascii_letters + string.digits, k=10))
        fullHash = generateHash(randomText.encode('utf-8'), algorithm)
        prefix = fullHash[:hexNum]

        if prefix in seen:
            originalText = seen[prefix]
            originalHash = generateHash(originalText.encode('utf-8'), algorithm)
            print("\nZnaleziono kolizję po " + str(attemps) + " próbach:")
            print("Tekst 1: " + originalText + " | Hash: " + originalHash)
            print("Tekst 2: " + randomText + " | Hash: " + fullHash)
            print("Wspólny prefix: " + str(prefix))
            break

        seen[prefix] = randomText

def toBits(data):  1 usage
    bits = np.unpackbits(np.frombuffer(data, dtype=np.uint8))
    return bits

def sha256Bits(data):  2 usages
    hashObj = hashlib.new('sha256')
    hashObj.update(data)
    return toBits(hashObj.digest())
```

```python
def testSAC_sha256():  1 usage
    outputBytesLength = 256
    changeCounter = np.zeros(outputBytesLength)

    for _ in range(1000):
        inputBytes = np.random.bytes(32)
        hashBits = sha256Bits(inputBytes)

        modifiedInput = bytearray(inputBytes)

        byteID = random.randint( a: 0,  b: 31)
        bitID = random.randint( a: 0,  b: 7)

        modifiedInput[byteID] ^= (1 << bitID)
        modifiedHash = sha256Bits(modifiedInput)
        changeCounter += (hashBits ^ modifiedHash)

    return changeCounter / 1000
```

```python
if __name__ == '__main__':
    algorithms = ['md5', 'sha1', 'sha224', 'sha256', 'sha384', 'sha512', 'sha3_224', 'sha3_256', 'sha3_384', 'sha3_512']
    userText = input('Podaj tekst: ')

    testData = ["Lorem ipsum dolor sit amet, consectetur efficitur.",
                "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras nec sollicitudin arcu. Curabitur sed sodales nunc, vel feugiat lectus. Nullam tristique scelerisque libero eu ultrices. Suspendisse proin.
                "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque consequat mauris sed mi mollis accumsan. Aenean fermentum commodo risus a venenatis. Duis ut elit urna. Praesent in malesuada nisl, et
                "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed tempus gravida dolor, vitae molestie tellus tempus et. Aliquam malesuada posuere enim viverra finibus. Duis elementum felis libero, ut dict

    for algorithm in algorithms:
        start = time.time()
        val = generateHash(userText.encode('utf-8'), algorithm)
        end = time.time()
        print("\nAlgorytm " + algorithm + ": \nTekst użytkownika o długości " + str(len(userText)) + " znaków: " + val + " | Czas działania: " + str(end - start) + " | Długość ciągu wyjściowego: " + str(len(val)))
        for data in testData:
            start = time.time()
            val = generateHash(data.encode('utf-8'), algorithm)
            end = time.time()
            print("Tekst testowy o długości " + str(len(data)) + " znaków: " + val + " | Czas działania: " + str(end - start) + " | Długość ciągu wyjściowego: " + str(len(val)))

    findCollision('sha256')
    prob = testSAC_sha256()
    meanProb = np.mean(prob)
    print("\nŚrednie prawdopodobieństwo zmiany dla wszystkich bitów na wyjściu po zmianie jednego bitu na wejściu: " + str(round(meanProb, 3)))
```

```
Podaj tekst: test1234

Algorytm md5:
Tekst użytkownika o długości 8 znaków: 10d7a4fca7442dda3ad93c9a726597e4 | Czas działania: 5.4597854614257 81e-05 | Długość ciągu wyjściowego: 32
Tekst testowy o wielkości 10MB: 96c001499aa1433b6fae1050106750bb | Czas działania: 0.010987758636647461 | Długość ciągu wyjściowego: 32
Tekst testowy o wielkości 100MB: f37972a28bc73b9f3905bb619fe6f047 | Czas działania: 0.11224913597106934 | Długość ciągu wyjściowego: 32
Tekst testowy o wielkości 500MB: 5fd543d0125bb4983521462276035dfc | Czas działania: 0.5571548938751221 | Długość ciągu wyjściowego: 32

Algorytm sha1:
Tekst użytkownika o długości 8 znaków: 9bc34549d565d9505b287de0cd20ac77be1d3f2c | Czas działania: 3.6001205444433594e-05 | Długość ciągu wyjściowego: 40
Tekst testowy o wielkości 10MB: cd13b61cad98dd2b1e14fba1d1d630ff9e177f7e | Czas działania: 0.08461935997009277 34 | Długość ciągu wyjściowego: 40
Tekst testowy o wielkości 100MB: d65ba3e6899e1f6950bb99a5722cfe375649ca4a | Czas działania: 0.8450763702392578 1 | Długość ciągu wyjściowego: 40
Tekst testowy o wielkości 500MB: 8566d3b067591ea96f5b924f591ab6f92d97c9d2 | Czas działania: 0.2239522933959961 | Długość ciągu wyjściowego: 40

Algorytm sha224:
Tekst użytkownika o długości 8 znaków: fe53be3576 3bccd7295b7225ed41f5c508440e8c4cc2ab52725105a | Czas działania: 3.1709671020587 81e-05 | Długość ciągu wyjściowego: 56
Tekst testowy o wielkości 10MB: d7eae606fbb5b9eb438a099e174c5d0cc38935bd16c01d897822c8a1 | Czas działania: 0.00538563728332 5195 | Długość ciągu wyjściowego: 56
Tekst testowy o wielkości 100MB: 1c421df5de10bb9f38181660fc35641557fb3132bc20989f44513fc0 | Czas działania: 0.04759764671325 6836 | Długość ciągu wyjściowego: 56
Tekst testowy o wielkości 500MB: 25da2a8741a5697418 20f2e00de11214c4f054c086b353241015 6489 | Czas działania: 0.2398252487182 6172 | Długość ciągu wyjściowego: 56

Algorytm sha256:
Tekst użytkownika o długości 8 znaków: 937e8d5fbb48bd4949536cd65b8d35c426b80d2f830c5c308e2cdec422ae2244 | Czas działania: 2.3841857910156 25e-05 | Długość ciągu wyjściowego: 64
Tekst testowy o wielkości 10MB: 1e8867b182396c9baf166a9616ed8e653131b40c82a300a0ad5f47fcb37908b9 | Czas działania: 0.0050940513610839 84 | Długość ciągu wyjściowego: 64
Tekst testowy o wielkości 100MB: 2e238537d35c1d2bddb702e4a831d11c94c7734696e30710bfb1a6017829af49 | Czas działania: 0.0486528873443603 5 | Długość ciągu wyjściowego: 64
Tekst testowy o wielkości 500MB: 5d0cfd9df1d516dcd372e08719fbca8ce08f6510b9a3eb8b4b6aa9a8044cfef | Czas działania: 0.2450318336468164 | Długość ciągu wyjściowego: 64

Algorytm sha384:
Tekst użytkownika o długości 8 znaków: 25421d7c7e6fd6aa71f707353cc3408fa1fb316e698136553b0e03eeae875a7808670e1083004822283 07a09504d7fdd4 | Czas działania: 4.0054321289062 5e-05 | Długość ciągu wyjściowego: 96
Tekst testowy o wielkości 10MB: abf4a8ee24e6ff77e7e8fdd1899de9a86deb2ac8f608b6e5aa98105adb5199df816568 0bbcc7630a937e12c989614c0f9 | Czas działania: 0.0899658960604453 12 | Długość ciągu wyjściowego: 96
Tekst testowy o wielkości 100MB: eca84a30705054ecbb1651efb5107edc53982aff9a5a3ed84ba7eb5a3c28451de38867ca17dc7beaeded24f40f95f66f | Czas działania: 0.1083245277404785 2 | Długość ciągu wyjściowego: 96
Tekst testowy o wielkości 500MB: ce449041785827ebcdd9e2dca01778d75071ec762dd2d9077850a20377682f165dd3e62cafd5c8f55e7bb5337835eda0 | Czas działania: 0.5341565608978271 | Długość ciągu wyjściowego: 96

Algorytm sha512:
Tekst użytkownika o długości 8 znaków: 2bbe0c48b91a7d1b8a6753a8b9cbe1db16b843791f3f91fe115621284df7a48f1cd71e9beb90ea614c7bd924250aa9e446a866725e685a65df5d139a5cd180dc9 | Czas działania: 2.4080276489257812e-05 | Długość ciągu wyjściowego: 128
Tekst testowy o wielkości 10MB: fdd1fc3d9d3d152db3a2b691765ad9bb51cf0626f074a598f306e942573c933f96dc973f362d72e75419adc89648c492aef884ebc1fbc4d040cb2b462fada932 | Czas działania: 0.0106914043424613 67 | Długość ciągu wyjściowego: 128
Tekst testowy o wielkości 100MB: e17c31175001c971695583 11c8ee06aa131413638b5088bfd053b61935c355d84ea411711abb1c943f3c200ef4d5e40dabc9b905f20e7476d1d8c77a6f458f | Czas działania: 0.1031837463378906 2 | Długość ciągu wyjściowego: 128
Tekst testowy o wielkości 500MB: a38dcb9e50d39193aec34fe5409dad7492e9c57a0c4cd071d695542d12acf5e996678d5d58554d1413f383155b85c787038a98a4bc520cc1d084f41fcaef365 | Czas działania: 0.5117537975311279 | Długość ciągu wyjściowego: 128

Algorytm sha3_224:
Tekst użytkownika o długości 8 znaków: 6f27da5c1d035d528c5517fb60431edc8b0cb343692da0c1977c7435 | Czas działania: 3.1948089599609375e-05 | Długość ciągu wyjściowego: 56
Tekst testowy o wielkości 10MB: e8abf590cc34b1623810551 56cfac3c83f008b5f3490026398b2da22 | Czas działania: 0.0165038108826836 | Długość ciągu wyjściowego: 56
Tekst testowy o wielkości 100MB: 7e37c998ce9470948433e3a2ac3e3459210ddfbdc39661260499 3ce5 | Czas działania: 0.1716182231903076 2 | Długość ciągu wyjściowego: 56
Tekst testowy o wielkości 500MB: ea2b23025396303 0d6506f09dae9557b9f9f64f8a0aa2d4be40e840 | Czas działania: 0.8536121845245361 | Długość ciągu wyjściowego: 56
```

```
Algorytm sha3_256:
Tekst użytkownika o długości 8 znaków: 0a8c939670358327a6bec55af0f44041dea04a3fffff2f48d0e165a969b8b9e7 | Czas działania: 2.3603439331054688e-05 | Długość ciągu wyjściowego: 64
Tekst testowy o wielkości 10MB: 503a00137b2e382d39448283615b93a91e3a3ee064df6f9a2d5408c24406d734 | Czas działania: 0.019356250762939453 | Długość ciągu wyjściowego: 64
Tekst testowy o wielkości 100MB: 1d1983f273346 3e916934f1c43d45b29518c9e56dde8400ed9b373ebdde301e0 | Czas działania: 0.18848705291748047 | Długość ciągu wyjściowego: 64
Tekst testowy o wielkości 500MB: dffc05f980221719cb92bc02bf67c9d68d8d684f9c102f300f9b8810320ea9f3 | Czas działania: 0.9262871742248535 | Długość ciągu wyjściowego: 64

Algorytm sha3_384:
Tekst użytkownika o długości 8 znaków: 831b07bde03dfc17a0a9bee562447eec7155c2c2ba9e71c2e5f7aed876540253af707b382862ea2128ffabb6fadf5e5e | Czas działania: 2.3841857910156 25e-05 | Długość ciągu wyjściowego: 96
Tekst testowy o wielkości 10MB: 13dc12b2a94051f5f819d3ad9f7fb1afe391b9b194ed91ee07392eada5b0fada549795980a5f65793b7eb7bf13ecb7f0 | Czas działania: 0.0234484672546386 72 | Długość ciągu wyjściowego: 96
Tekst testowy o wielkości 100MB: bfa38af8ff7544b4ae22872d2a082ba44b6faf5193573ca4bc9e33f6ce8bc807eb125f80222207d448d7e82794d7982a | Czas działania: 0.2343664169311523 4 | Długość ciągu wyjściowego: 96
Tekst testowy o wielkości 500MB: cc4f58d48b4393ab9181a18de4ea1f9dd67bef7cfd93293ae5f82a316abba1394a09976294a280325e6b81879df27d53 | Czas działania: 1.1611483097076416 | Długość ciągu wyjściowego: 96

Algorytm sha3_512:
Tekst użytkownika o długości 8 znaków: c9a9d2e15311717e61ef5a916e0800271a619edaed0fe0eada11e68e3c316bef64be59ae9b1a9c9cfaaf1851ae71b20746d8f8599bca29931cb5e6983b7029d9 | Czas działania: 2.5749206542968 75e-05 | Długość ciągu wyjściowego: 128
Tekst testowy o wielkości 10MB: 45fb302e9385a044e24e7adc390a9ca4bd4aa2dec8e502fb84cbf7db38f5c439855ac6bac3204e594a2cc8c267005254815257 56cad6a785eefa77902b99c3e2 | Czas działania: 0.0318713188171386 7 | Długość ciągu wyjściowego: 128
Tekst testowy o wielkości 100MB: 7fe2932ef41ff4733daca518bc7e12287d54353152c5bb449e413c27de5637b64f3130884e494631beddc0d4a3c2fc0b444c037b0da23bccfcda0b7a65390bf4 | Czas działania: 0.3263719081878662 | Długość ciągu wyjściowego: 128
Tekst testowy o wielkości 500MB: 4112dcc6a49c55da8a19cf7cd26ef87ce49bc6cdd091d267858 5e47def1569b3163251034fbb6e3c57b765caac640f332b2d5362adf8db98284b27ea5e796343 | Czas działania: 1.6485976802029419 | Długość ciągu wyjściowego: 128

Znaleziono kolizję po 1 próbach:
Tekst 1: 0KFpWV63W0 | Hash: 233da32a8cc37bec3f3f330d5795e510a819b01fa6be78eac2f4458e3ee370a8
Tekst 2: nRMKFV8MBb | Hash: 233dded512043c2655ea95052b4cf70b00499de9bb890fdfb17594848cef3df9
Wspólny prefix: 233

Średnie prawdopodobieństwo zmiany dla wszystkich bitów na wyjściu po zmianie jednego bitu na wejściu: 0.5
```

2. **Omówienie sposobu implementacji.**
   Aplikacja została zaimplementowana w języku Python z użyciem bibliotek:
   - hashlib, zawierającej natywny dostęp do algorytmów kryptograficznych
   - time, pozwalającej na zmierzenie czasu działania algorytmów
   - random, pozwalającej na wygenerowanie danych do testów oraz wybranie losowych indeksów
   - string
   - numpy

   Do generowania skrótów użyłam funkcji hashlib.new(), która pozwala na dynamiczny wybór algorytmów.

   W badaniu kolizji wykorzystałam słownik, w którym zapisywane są 12-bitowe prefiksy oraz oraz odpowiadający im losowo wygenerowany tekst. Sprawdzam, czy prefiksy występują wielokrotnie, co oznacza kolizję.

   Do zbadania własności SAC generowane jest 256 bitów oraz funkcja skrótu. Następnie zmieniany jest jeden losowy bit w oryginalnym ciągu (XOR) i ponownie generowany jest skrót. Badana jest zmiana bitów między skrótami z użyciem XOR-a oraz obliczone zostaje prawdopodobieństwo zmiany.

3. **Określenie roli soli w tworzeniu skrótów.**
   Sól to losowy ciąg danych dodawany do hasła przed procesem hashowania. Zapewnia to unikalną zmianę wyniku, dzięki czemu nawet identyczne hasła mają różne skróty. Zwiększa to bezpieczeństwo i zapobiega łamaniu haseł.

4. **Na podstawie powszechnie dostępnych źródeł odpowiedz na pytanie – czy funkcję MD5 można uznać za bezpieczną? Czy dotychczas zostały znalezione dla niej jakiekolwiek kolizje?**
   Funkcja MD5 nie może być uznana za bezpieczną. Jest mało odporna na ataki oraz podatna na kolizje, które zostały znalezione także w moich testach.

5. **Zestawienie wyników wraz ze stosownymi wnioskami.**
   1) Porównanie algorytmów

Algorytm	**Długość na wyjściu [w bitach]**	**Czas działania (10 MB) [s]**	**Czas działania (100 MB) [s]**	**Czas działania (500 MB) [s]**
MD5	128	0.01084	0.11019	0.55026
SHA-1	160	0.00471	0.04316	0.21801
SHA-2 (224)	224	0.00476	0.04593	0.23296
SHA-2 (256)	256	0.00512	0.04599	0.23507
SHA-2 (384)	384	0.01286	0.09806	0.49419
SHA-2 (512)	512	0.00980	0.09858	0.49447
SHA-3 (224)	224	0.01587	0.15828	0.79466
SHA-3 (256)	256	0.01762	0.17777	0.88169
SHA-3 (384)	384	0.02164	0.21949	1.09639
SHA-3 (512)	512	0.03198	0.31633	1.58012

   2) Kolizje
      Znaleziono kolizję po 1 próbach:
      Tekst 1: 7G9tmmpGL8
      Hash:
      7be0e08890612812d111c2dda31947151ecb1e418042518a2c7feb478c5c0e7b

      Tekst 2: LF8HZgKRhS
      Hash:
      7be4971570be5267513e421dec7cdaf619f29eaba7f3df0c1266df22559657dd

      Wspólny prefix: 7be

   3) SAC
      Średnie prawdopodobieństwo zmiany dla wszystkich bitów na wyjściu po zmianie jednego bitu na wejściu: 0.5

Wnioski:

Niezależnie od długości tekstu wejściowego (od 10 do 500 MB), dany algorytm zawsze generuje skrót o identycznej liczbie znaków. Czas obliczeń rośnie wraz ze wzrostem objętości danych. Algorytmy z rodziny SHA-3 wymagają nieco więcej operacji, co jest szczególnie widoczne przy przetwarzaniu większych bloków danych. Im dłuższy ciąg wyjściowy, tym trudniejsze jest znalezienie kolizji, co bezpośrednio przekłada się na wyższy poziom bezpieczeństwa kryptograficznego